

Chapter 17

Transactions



- Transaction State
- Implementation of Atomicity and Durability
- Concurrent Executions
- Serializability
- Recoverability
- Implementation of Isolation
- Transaction Definition in SQL
- Testing for Serializability.



- The term **transaction** refers to a collection of operations that form a single logical unit of work.
- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.



- E.g. transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions



- **Atomicity requirement**

if the transaction fails after step 3 and before step 6,
money will be “lost” leading to an inconsistent database
state

- ▶ Failure could be due to software or hardware

the system should ensure that updates of a partially
executed transaction are not reflected in the database

- **Durability requirement** — once the user has been notified
that the transaction has completed (i.e., the transfer of the
\$50 has taken place), the updates to the database by the
transaction must persist even if there are software or
hardware failures.



- **Consistency requirement** in above example:

the sum of A and B is unchanged by the execution of the transaction

- ✓ A transaction must see a consistent database.

- ✓ During transaction execution the database may be temporarily inconsistent.

- ✓ When the transaction completes successfully the database must be consistent

- ▶ Erroneous transaction logic can lead to inconsistency



Transaction Example(3)

• **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

- 1. **read**(A)
- 2. $A := A - 50$
- 3. **write**(A)
- 4. **read**(B)
- 5. $B := B + 50$
- 6. **write**(B)

T2

read(A), read(B), print($A+B$)

- Isolation can be ensured trivially by running transactions **serially**
 - that is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.



Atomicity (原子性) : Either all operations of the transaction are properly reflected in the database or none are. **Transaction-management component**

Consistency (一致性) : Execution of a transaction in isolation preserves the consistency of the database.

Isolation (隔离性) : each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.

Concurrency-control schemes

Durability (保持性) : After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

recovery-management component



- **Active**

the initial state; the transaction stays in this state while it is executing

- **Partially committed**

after the final statement has been executed.

- **Failed**

after the discovery that normal execution can no longer proceed.

- **Aborted**

Two options after it has been aborted:

restart the transaction – only if no internal logical error

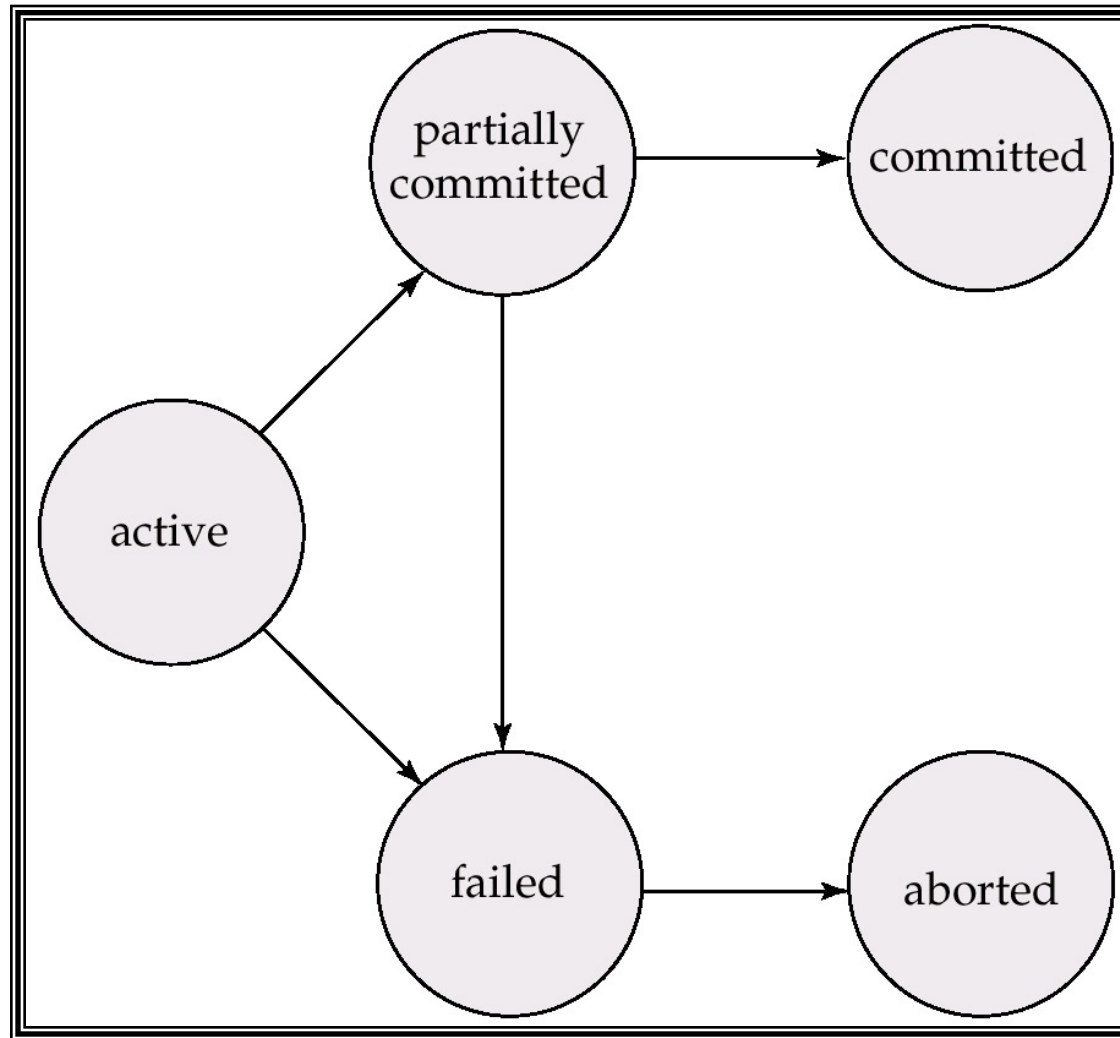
kill the transaction

- **Committed**

after *successful completion*.



Transaction State



Why Concurrent Executions *DataBase System Concepts*

- **increased processor and disk utilization**, leading to better transaction *throughput*(吞吐量) : one transaction can be using the CPU while another is reading from or writing to the disk
- **reduced average response time** for transactions: short transactions need not wait behind long ones.

Concurrency control schemes – mechanisms to achieve isolation to prevent them from destroying the consistency of the database.



- *Schedules* – a sequences that specify the chronological order in which instructions of concurrent transactions are executed.
 - a schedule for a set of transactions must consist of all instructions of those transactions.
 - must preserve the order in which the instructions appear in each individual transaction.
- **serial schedule(串行调度)**
- **concurrent schedule (并发调度)**



T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>



- Basic Assumption – Each transaction preserves database consistency.
- Thus serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.
 - conflict serializability
 - view serializability



Conflict Operations:

1. $l_i = \text{read}(Q)$, $l_j = \text{write}(Q)$. They conflict.
 2. $l_i = \text{write}(Q)$, $l_j = \text{read}(Q)$. They conflict
 3. $l_i = \text{write}(Q)$, $l_j = \text{write}(Q)$. They conflict
 4. $l_i = \text{read}(Q)$, $l_j = \text{read}(Q)$. l_i and l_j don't conflict
 5. Operations on different data items, don't conflict
- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
 - We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.



T_1	T_2
read(A) write(A)	read(A) write(A)
read(B) write(B)	
	read(B) write(B)

T_3	T_4
read(Q)	write(Q)
write(Q)	



● *Testing for Conflict Serializability* DataBase System Concepts

● **Precedence graph :**

● $G(V,E)$, where V is a set of vertices and E is a set of edges. The set of vertices a direct consists of all the transactions participating in the schedule. the set of edges consists of all the edges $T_i \rightarrow T_j$ for which one of three conditions holds:

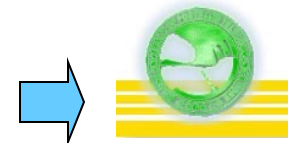
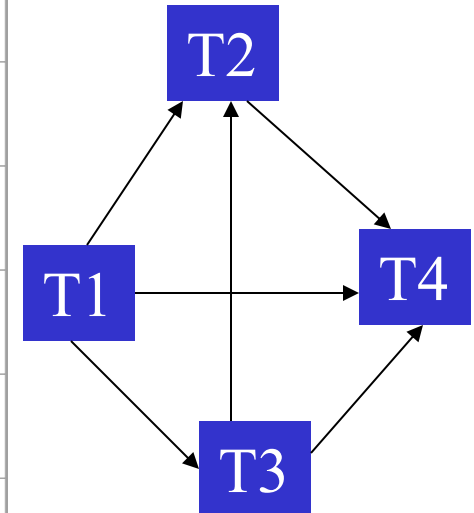
- 1 T_i executes write(Q) before T_j executes read(Q);
- 2 T_i executes read(Q) before T_j executes write(Q);
- 3 T_i executes write(Q) before T_j executes write(Q);

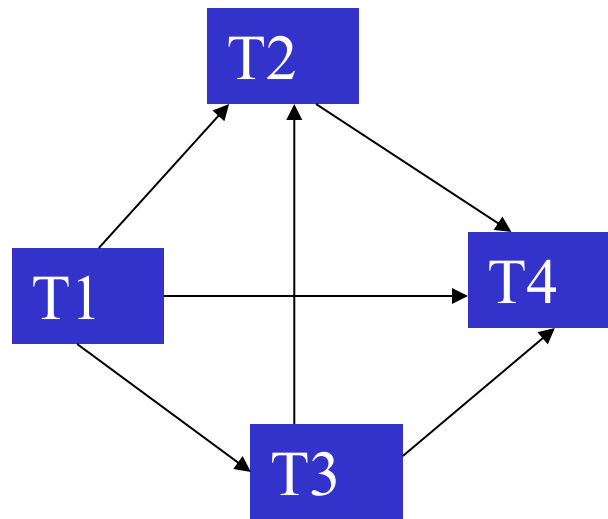
● *A schedule is conflict serializable if and only if its precedence graph is acyclic.*



If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph. This is a linear order consistent with the partial order of the graph.

T1	T2	T3	T4
		Write(Y)	
Read(X)			
	Read(Y)		
		Write(X)	
	Write(X)		
		Write(Z)	
			Read(Z) Write(X)





$T1 \rightarrow T3 \rightarrow T2 \rightarrow T4$



T1	T2	T3	T4
	read(A)		
read(A)			read(B)
read(C)		read(B)	
		write(B)	
			read(A)
	write(A)		
write(C)			
	write(C)		
			write(A)
			write(B)

Recoverability

Recoverable schedule — if a transaction T_j reads a data items previously written by a transaction T_i , the commit operation of T_i appears before the commit operation of T_j .

The following schedule (Schedule 11) is not recoverable if T_9 commits immediately after the read

T_8	T_9
read(A)	
write(A)	
	read(A)
read(B)	



Cascading rollback – a single transaction failure leads to a series of transaction rollbacks

T_{10}	T_{11}	T_{12}
read(A) read(B) write(A)	read(A) write(A)	read(A)

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

Can lead to the undoing of a significant amount of work.



- **Cascadeless schedules** — cascading rollbacks cannot occur; for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless



- A database must provide a mechanism that will ensure that all possible schedules are
 - either conflict or view serializable
 - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
 - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.



Concurrency Control Protocol

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.
- Tests for serializability help us understand why a concurrency control protocol is correct.



- **Serializable (可串行化)** — 隔离最高级别
- **Repeatable read(可重复读)** — only committed records to be read. 在开始读取数据 (事务开启) 时, 不再允许修改操作
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed (已提交读)** — only committed records can be read. 一个事务要等另一个事务提交后才能读取数据。
 - Successive reads of record may return different (but committed) values.
- **Read uncommitted (未提交读)** — even uncommitted records may be read.



脏读: 读到了别的事务回滚前的脏数据。

不可重复读: 当前事务先进行了一次数据读取, 然后再次读取到的数据是别的事务修改成功的数据, 导致两次读取到的数据不匹配。

幻读: 事务A首先根据条件查询得到N条数据, 事务B改变了这N条数据之外的M条或者增添了M条符合事务A查询条件的数据, 导致事务A再次搜索发现有N+M条数据了, 就产生了幻读。

当前事务读第一次取到的数据比后来读取到数据条目不一致。



四种隔离级别可能导致的问题:

Serializable (可串行化): 最严格的级别，事务串行执行，资源消耗最大;

REPEATABLE READ(重复读): 避免了“脏读取”和“不可重复读取”的情况，但不能避免“幻读”。

READ COMMITTED (提交读): 避免了“脏读”，但不能避免“幻读”和“不可重复读取”。

Read Uncommitted(未提交读): 会导致“脏读”、“幻读”和“不可重复读取”。



- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
 - ★ **Commit work** commits current transaction and begins a new one.
 - ★ **Rollback work** causes current transaction to abort.



T_1	T_2	T_3
	read(Y)	
read(X)		
	$Y := Y - 20$	
	write(Y)	
$X := X + 10$		
write(X)		read(X)
	read(X)	
	$X := X - 10$	
commit		
	write(X)	
	commit	
		read(Y)
		$X := X + Y$
		write(X)
		commit

Is S a recoverable schedule? and why?

Is S a cascadeless schedule? and why?

T ₁	T ₂	T ₃	T ₄
begin_transaction			
	begin_transaction		
update <i>Student</i> set <i>stuName</i> ='Li' where <i>stuID</i> =10			
		begin_transaction	
	update <i>Student</i> set <i>age</i> = <i>age</i> +1 where <i>stuID</i> =20		
		select <i>age</i> from <i>Student</i> where <i>stuID</i> =50	
			begin_transaction
			update <i>Student</i> set <i>height</i> = <i>height</i> +5 where <i>stuID</i> =30
update <i>Grade</i> set <i>grade</i> = <i>grade</i> +1 where <i>stuID</i> =40 AND <i>CosID</i> =10			
	select <i>stuName</i> from <i>Student</i> where <i>stuID</i> =10		
	commit		
commit			
			select <i>stuID</i> , <i>grade</i> from <i>Grade</i> where <i>stuID</i> =40 AND <i>CosID</i> =10
			commit
		Select <i>height</i> from <i>Student</i> where <i>stuID</i> =30	
		commit	

- (1) Construct the precedence graph for S.
- (2) Is S a serializable schedule? If not, give the reason. If it is, give a serial schedule that is equivalent to S.
- (3) Is S a recoverable schedule, and why?
- (4) Is S a cascadeless schedule, and why?